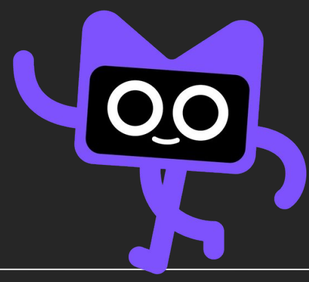
CREATIVE COMPUTING LEVEL 5

Mobile App Development

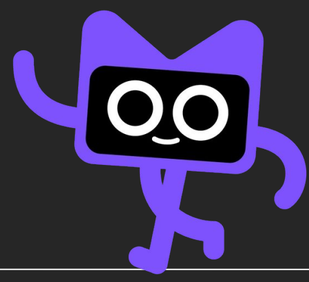
Kotlin Crash Course for
Android App Development

What Is Kotlin?



- Kotlin is a modern programming language used to create Android apps.
  - Developed by JetBrains
  - Official Android language (Google-approved)
  - Runs on the Java Virtual Machine (JVM)

Why Kotlin?



Java

```
String name = "Mack";
System.out.println(name);
```

Kotlin

```
val name = "Mack"
println(name)
```

Kotlin Environment Checklist

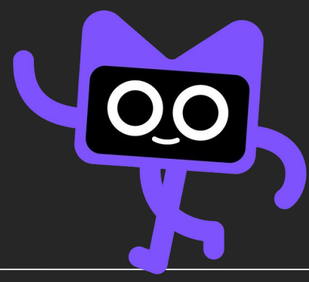
- Visual Studio Code
- Java Development Kit (JDK)
- Kotlin Compiler
- Kotlin Extension for VS Code
  - Kotlin (fwcd)

CREATIVE COMPUTING LEVEL 5

Coding in Kotlin

Kotlin Program Structure

Kotlin Program Structure

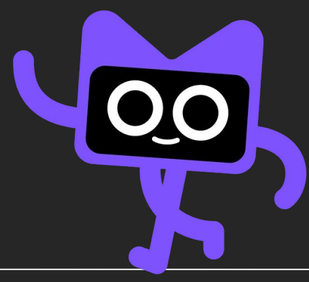


Main.kt

```
fun main() {
  println("Hello, Kotlin!")
}
```

- .kt → Kotlin file extension
- fun → keyword to create a function
- main → program starting point
- { } → code block

Output in Kotlin



Kotlin

```
println("Hello")
```

Java

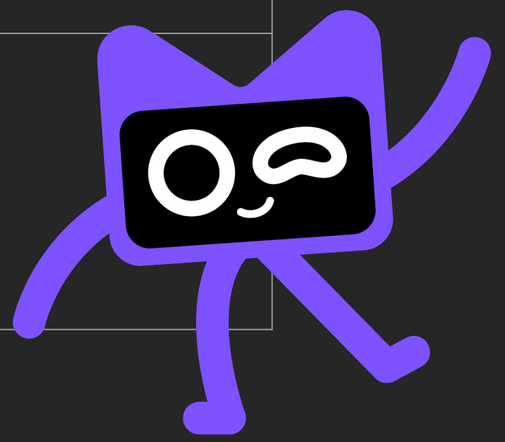
```
System.out.println("Hello");
```

Python

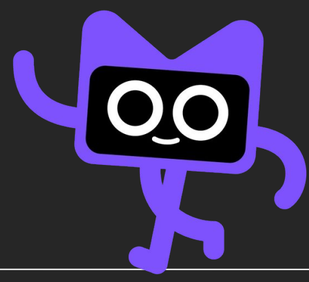
```
print("Hello")
```

Activity

Level 1	Print a short greeting that includes your seatmate’s name.
Level 2	Print a list of things you enjoy doing, each on its own line.
Level 3	Create a simple design or shape using symbols.



Comments in Kotlin



- Comments are notes in code that are ignored by the computer but help humans understand the program.

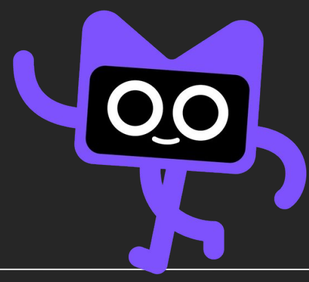
```
// This is a single-line comment


/*
This is a multi-line comment
*/
```

CREATIVE COMPUTING LEVEL 5

Coding in Kotlin

Variables, Data Types, and Operators

Variables: val and var



- Variables are containers that store data
- val → read-only (constant)
- var → changeable

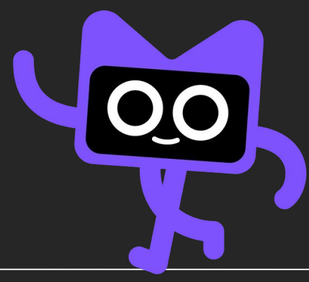
```
val name = "Mack"  // cannot change
var age = 16       // can change
println(name+" is "+age+" years old")
println("$name is $age years old")
```

Data Types

- Data types define the kind of data a variable can store

```
val name: String = "Mack"
val age: Int = 16
val height: Double = 1.75
val isStudent: Boolean = true
```

Operators



Arithmetic

```
+  -  *  /  %
```

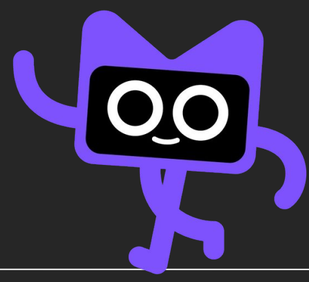
Comparison

```
==  !=  >  <  >=  <=
```

Logical

```
&&  ||  !
```

Getting Input



- readln() → get input

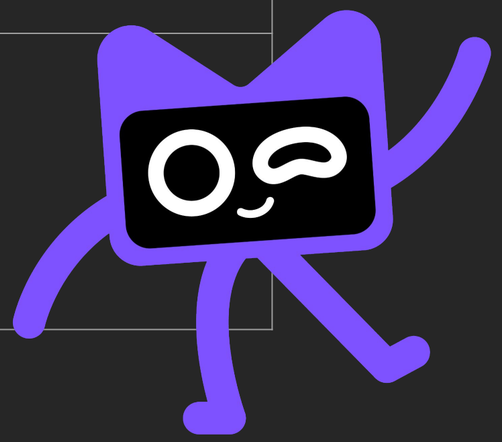
```
val name = readln()
println("Hello $name")
```

- toInt() → convert string to int

```
val age= readln().toInt()
println("$age years old")
```

Activity

Level 1	Ask the user for their name and print a greeting.
Level 2	Ask the user for three pieces of information, store them in variables, and print them back in a formatted way.
Level 3	Ask the user for two numbers and print the sum, difference, product, and quotient.

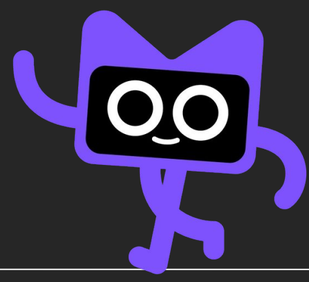


CREATIVE COMPUTING LEVEL 5

Coding in Kotlin

Conditional Control Structure

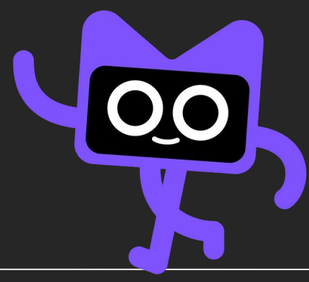
if Statement



- Executes a block of code only if a condition is true.

```
if (score > 40) {
    println("Pass")
}
```

if-else Statement



- Provides an alternative path if the condition is false.

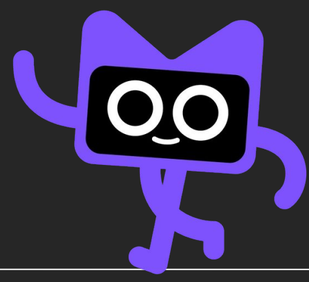
```
if (temperature > 30) {
    println("Hot")
} else {
    println("Not hot")
}
```

if-elseif-else Statement

- Checks multiple conditions in sequence.

```
if (number < 0) {
    println("Negative")
} else if (number > 0) {
    println("Positive")
} else {
    println("Zero")
}
```

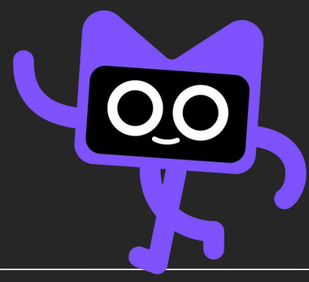
when Statement



- Chooses an action based on matching values or conditions.
- Syntax:

```
when (value) {
    case1 -> action1
    case2 -> action2
    else -> defaultAction
}
```

when Statement



- Simple example:

```
when (rating) {
    5 -> println("Excellent")
    4 -> println("Good")
    3 -> println("Fair")
    else -> println("Needs Improvement")
}
```

when Statement

- Condition-based:

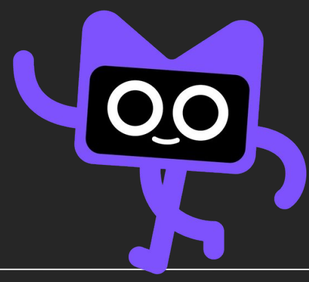
```
when (number) {
    in 1..5 -> println("Low range")
    in 6..10 -> println("Mid range")
    else -> println("Out of range")
}
```

when Statement

- Condition-based:

```
when {
    number < 0 -> println("Negative")
    number > 0 -> println("Positive")
    else -> println("Zero")
}
```

when Statement

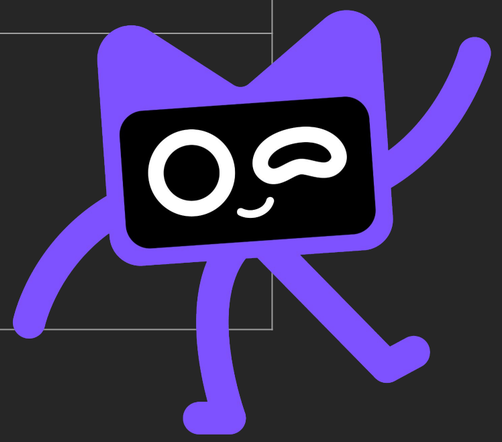


- As an expression:

```
val result = when (number) {
    in 1..10 -> "Small"
    in 11..20 -> "Medium"
    else -> "Large"
}
```

Activity

Level 1	Using *if-else*, ask the user if they are hungry (or). Print a message based on their answer.
Level 2	Using *nested if-else*, ask the user what meal of the day it is (breakfast, lunch, or dinner). Print a suggestion.
Level 3	Using *when* ask the user for a number from 1 to 7 and print the day of the week.

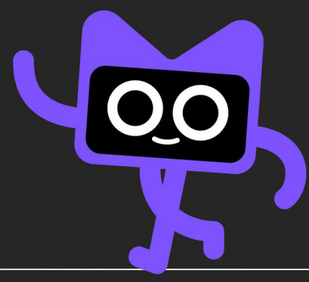


CREATIVE COMPUTING LEVEL 5

Coding in Kotlin

Repetition Control Structure

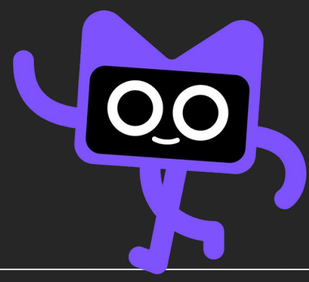
for Loop



- Used when you know exactly how many times to repeat.

```
for (i in 1..5) {
    println(i)   // 1, 2, 3, 4, 5
}
```

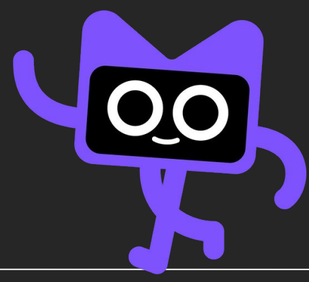
for Loop



- Using **until** stops before the end value.

```
for (i in 1 until 5) {
    println(i)   // 1, 2, 3, 4
}
```

for Loop



- Using **downTo** counts backwards.

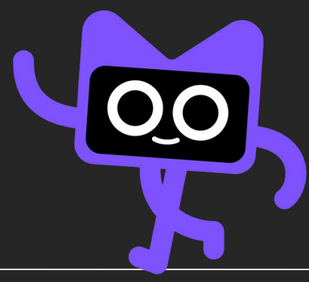
```
for (i in 5 downTo 1) {
    println(i)   // 5, 4, 3, 2, 1
}
```

for Loop

- Using **step** changes the increment.

```
for (i in 1..10 step 2) {
    println(i)   // 1, 3, 5, 7, 9
}
```

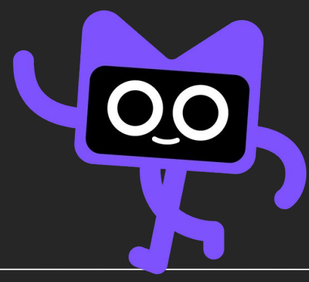
while Loop



- Used when the repetition depends on a condition.

```
var count = 5


while (count > 0) {
    println(count)
    count--
}
```

do-while Loop

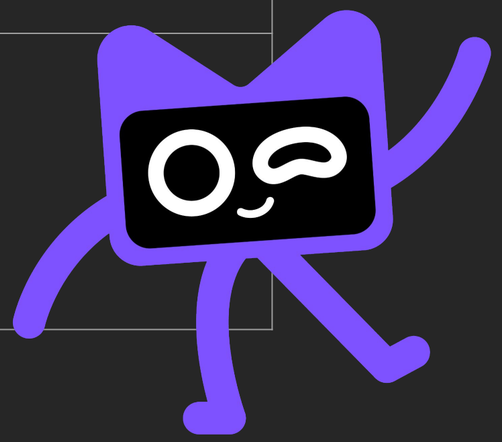


- Similar to while, but guarantees at least one execution.

```
var number = 1


do {
    println(number)
    number++
} while (number <= 3)
```

Activity

Level 1	Using a for loop, simulate an elevator going from floor 1 to floor 12, but print “Skipping 4 (maintenance)” instead of printing the number 4.
Level 2	Using a for loop, simulate a battery draining from 100% to 0%, decreasing by 7% each step, but stop once it hits below 15% and print “Battery critical”.
Level 3	Using *while/do-while loop,* keep asking the user to guess a secret number until they get it right. There should also be a limit to the number of tries.



CREATIVE COMPUTING LEVEL 5

Coding in Kotlin

Functions

Functions

- Reusable blocks of code that perform a task.

```
fun greet() {
    println("Hello!")
}


greet()
```

Functions

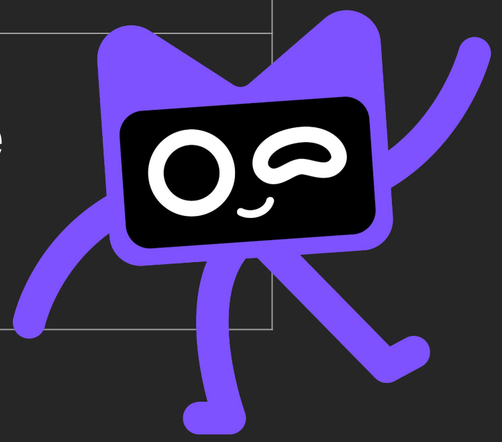
- Function with parameters:

```
fun greet(name: String) {
    println("Hello, $name")
}


greet("Sam")
```

Functions

- Function that returns a value:

```
fun add(a: Int, b: Int): Int {
    return a + b
}


val result = add(6, 7)
```

Activity

Level 1	Ask the user to press Enter to “start the game,” then call a function that prints a starting message.
Level 2	Ask the user for their character name and pass it to a function that prints a custom intro.
Level 3	Ask the user for attack and defense values, return the damage, and print it.